

Code optimization and transformation

Indice

Immediate representations	2
IR Trees	3
Basic blocks	3
Control flow graph (CFG)	3
Static single assignment	4
Instruction selection	5
Pattern matching	6
Other code generation approaches	6
Data-flow analysis	6
Liveness analysis	7
Generalization	8
Reaching definitions	9
Register allocation	9
Graph coloring approach	9
Linear scan	10
Register allocation with SSA	11
Optimization	11
Correctness	11
Target architecture's influence	12
Optimization phases	12
Dependence analysis	12
General purpose transformations	14
Data-flow based loop transformations	14
Loop reordering	14
Loop restructuring	15
Loop replacement transformations	16
Memory access transformations	16
Partial evaluation	17
Redundancy elimination	18
Procedure call transformation	18
Scheduling	19
Basic blocks	19
Beyond basic blocks	19
Trace scheduling	20
Superblock scheduling	20
Modulo scheduling	20
LLVM framework	20
Pass overview	21
The IR	21

The CFG	23
Basic blocks	23
Instructions	24
Data flow	24
Normalization passes	24
Variable promotion (mem2reg)	24
Loops	25
Loop simplify (loop-simplify)	25
Loop-closed SSA (lcssa)	25
Induction variable reduction (indvars)	25
Analysis passes	25
Control flow graph	26
Dominator tree (domtree) and post-dominator tree (postdomtree)	26
Loop information (loops)	26
Scalar evolution (scalar-evolution)	26
Alias analysis	27
Memory SSA (memoryssa)	28
Linker	28
ELF crash-course	28
Symbols	29
Relocations	29
Linkers more in depth	29
Memory mapping and segments	30
Static linking	30
Dynamic linking	30
Global Offset Table and Procedure Linkage Table	31
.dynamic section and stripping	31
Advanced linking features	32
DWARF debugging standard	32
Describing data	32
Describing code	33
Other information	33

Immediate representations

A compiler is split in two:

1. The **front end** handles lexical/syntactic analysis and semantic checks
2. The **back end** generates code, allocates registers and applies target-dependent optimization

The **front end generates an intermediate representation (IR)** and the backend operates on that. An IR is **composed of**:

1. **Intermediate language**: a functional representation of the source
2. **Metadata**: information about the source code useful for optimization

There are **3 main types** of IR:

- **Expression trees**
- **Three address instructions**: pseudo assembly languages (register based)
- **VM bytecode**: like java bytecode or DotNet CIL (usually a stack-based language)

An IR, ideally, needs to be **convenient to be produced during semantic analysis and to translate into machine code** (first conflict: one mandates that it should be similar to the source language while the other similar to assembly); moreover **each construct must have a simple and clear meaning**, such that we can easily specify and implement optimizing transformations. For interpreted IRs, other requirements may arise.

IR Trees

IR constructs are **nodes of a tree**, where **child nodes contain the operands** of the parent node. **Each IR construct must describe very simple operations** such as fetch/store/add/jump.

We will consider the following node types:

1. **CONST(i)** integer constant
2. **NAME(n)**: symbolic constant
3. **TEMP(t)**: a temporary to be later mapped to a register
4. **BINOP(o, e1, e2)**: binary operator applied to operands, each obtained by evaluating the operand expressions in order
5. **MEM(e)**: content of a word, starting at address e
6. **CALL(f, l)**: call function with given argument list

... and the following statements:

1. **MOVE(temp T, e)**: evaluate e and move the result into temporary t
2. **MOVE(MEM(e1), e2)**: evaluate e1 and e2 and move the result into memory at address e2
3. **JMP(l)**: unconditional jump
4. **CJUMP(o, e1, e2, t, f)**: evaluate the two expressions, compare the results with operator o; if true jump to t, otherwise to f
5. **LABEL(N)**: define symbolic constant as the current code address

Example code:

```
a = b if x < 5 else 7
```

Translated into our IR:

```
          CJUMP(LT, TEMP x, CONST 5, NAME t, NAME f)
LABEL t   MOVE (TEMP a,TEMP b)
          JMP (NAME out)
LABEL f   MOVE (TEMP a,CONST 7)
LABEL out
```

In our expression tree representation, we need to **introduce a SEQ pseudo node that encodes the sequential nature of the program**.

Basic blocks

A basic block is defined as a **sequence of statements that is always entered at the beginning and exited at the end**. The **first statement should be a LABEL** and the **last statement a JUMP/CJUMP**, with **no other label or jump in between**.

To **construct basic blocks** from generic IR code we can apply the following algorithm:

- **Scan** the statement sequence
 - **When LABEL is found, start a new block**
 - **When a jump is found, end the current block and start a new one**
- Any **block opened with no label gets a new one**
- Any **block closed with no jump receive a jump to the next block label**
- A **final block with just a label statement is inserted at the end**

Basic blocks are very important since they are self contained and can be relocated however we please.

Control flow graph (CFG)

It is a **directed graph** such that:

1. There is **one node i** for **each IR statement (stat_i)**
2. There are **two additional nodes i_in** and **i_out**
3. There is **one edge (i, i')** if **stat_i' is executed immediately after stat_i**
4. Each node must **have at most two immediate successors**
5. For the **first statement (stat_0)** there is an **arc (i_in, i_0)**

6. An arc (j, i_{out}) is added **for each node j** bound to a statement `stat_j` **preceding an exit point** of the program

A basic block is recognized as a sequence of nodes $\langle i_0, \dots, i_n \rangle$ such that:

1. There are arcs $(i_0, i_1), \dots, (i_{n-1}, i_n)$
2. **No other arc** in the CFG **has any node of the sequence except i_0** as its **target**
3. **No other arc** in the CFG **has any node of the sequence except i_n** as its **starting node**

CFGs are very useful in detecting loops. **Loops in a CFG appear as a directed cycle** in the CFG. A loop is a **strongly connected component**, which is defined as:

A set of nodes such that **each one is reachable from every other node** in the set.

Definition (Dominance relation):

A node d dominates a node n if m occurs before n on every directed path from the start node s_0 to n . Moreover every node dominates itself.

Definition (Immediate dominator):

A node $m \neq n$ is an immediate dominator of n if

$$m \text{ dom } n \wedge (d \text{ dom } m \forall d : d \text{ dom } n)$$

Static single assignment

Many optimizations require to represent definitions and uses of variables. One possibility is to maintain an explicit structure representing all information about variables; an improvement is to put code into “static single assignment form” (SSA). **SSA is an IR where each variable has only one definition, but this definition can be in a loop.**

Non SSA

`a = x + y`

`b = a - 1`

`a = y + b`

SSA

`a1 = x + y`

`b1 = a1 - 1`

`a2 = y + b1`

The **advantages** of SSA are:

1. It **simplifies data-flow analysis** and program **optimizations**
2. **Reduce the space and time needed to represent definitions and uses**
 - Assume that N uses M definitions in about $N + M$ instructions, then an explicit structure for definitions and uses is $\mathcal{O}(MN)$ while SSA is $\mathcal{O}(M + N)$
3. **Simplifies register allocation**
4. **Unrelated uses** of the same variable **disappear**

To **convert a basic block into SSA** we have that:

1. Each **new definition of a variable a** is modified to **define a fresh variable a_1, a_2 , etc...**
2. Each **use of the variable is changed to use the most recently defined version**

The second point poses a problem: **when a statement has more than one predecessor** (e.g. because of two conditional branches converging), **the idea of “most recent definition” becomes nonsense** since the definition used to assign the new value depends on the control flow. We thus **introduce the ϕ -function**, a **special notation that denotes a runtime choice of the predecessor**. The ϕ -function has as many arguments as predecessors. To know which edge was taken, if the program has to be made executable, we **place a MOVE instruction on each incoming edge**; during compilation **data-flow analysis will provide the connection** between a use a_i and a definition a_j .

Dominance properties:

1. If x has the j -th argument of a ϕ -function in block n , then the **definition** of x is **always executed before** (pre-dominates) the j -th predecessor of n
2. If x is **used in a non- ϕ -statement** in block n , then the **definition** of x **must dominate** n

ϕ -function conversion algorithm outline:

0. Hypothesis: the start node contains an implicit definition of every variable
 1. Inserting ϕ functions
 2. Assigning subscripts

We know already how to do the second point. The first one is more difficult. One way to do it is verify the **path-convergence criterion**:

Place $a = \phi(\dots)$ at node z if all conditions are met:

1. There exist two basic blocks such that $x : a = \dots$ and $y : a = \dots$
2. There exist two non-empty paths P_{xz} from x to z , and P_{yz} from y to z such that:
 1. P_{xz} and P_{yz} do not have any node in common except z
 2. Node z does not occur within both P_{xz} and P_{yz} prior to the end

The algorithm then becomes:

```
while exists blocks x,y,z staisfying PCC and z does not contain a phi for a
  insert "a = phi(a, a, ldots, a)" at z
... # relabel
```

This algorithm is **simple**, but **very slow** since verifying the PCC implies verifying two existential quantifiers over all of the instructions space.

Another way is to leverage the dominance properties of SSA. First let us introduce some **preliminaries**:

1. Node x **strictly dominates** node w if x dominates w and $x \neq w$
2. Predecessor/successor denote CFG relations
3. Parent/child denote dominance tree relations
4. In a tree, an **ancestor** of node n is the parent of n or the parent of an ancestor of n
 - An ancestor is proper if it is not the parent

Definition (Dominance frontier):

The dominance frontier $DF(x)$ of a node x is the set of all nodes w such that x dominates a predecessor of w , but does not strictly dominate w

If x dominated all predecessor of w it would dominate w too.

Intuitively, DF is the border between dominated and non-dominated nodes, therefore it is a point of convergence of disjoint paths.

To **implement the algorithm using DF**, for each variable defined in node x we **insert** a ϕ function in **every node that is in** $DF(x)$. This is **equivalent to the path convergence criterion**:

1. If a_x is the i -th argument of a ϕ function in block z , then the definition $x : a = \dots$ dominates the i -th predecessor of z
2. If a is used in a non- ϕ statement in block n , then the definition of a dominates n

Since a ϕ function is a kind of definition, we must **apply the criterion to each newly introduced** ϕ functions. This requires an **iterative procedure** that terminates when no nodes need new ϕ functions.

Instruction selection

After we have our IR, we need to **generate instructions for said IR**. The **mapping** between the two **can vary depending on CPU ISA** type:

- **RISC** architectures usually have a **one-to-one or one-to-many** relation between IR nodes and target instructions
 - Instructions are very simple and often we need many instructions to do a single operation

- CISC architectures usually **may also have a many-to-one** relation
 - Instructions are complex and can do multiple operations

We have **two main strategies**:

1. **Tabular**: we have a lookup table where each node is translated to snippet of machine code
 - **Very simple** method
 - Works **very well for RISC** targets where we have one-to-one or one-to-many
2. **Pattern matching** algorithms
 - More **complicated**
 - Works very well for CISC targets where we have many-to-one

Since the tabular method is trivial to implement, we will outline the pattern matching one.

Pattern matching

The main idea is the following:

1. **For each IR instruction**, we define a **subtree-pattern** (called tile) and a relative output instruction
2. We apply a **tree-covering algorithm**

This way we can **reduce** the problem to a **minimum-cost tree-covering**. The **cost criterion depends on our application** requirements (execution time, energy, code size etc...).

There are many algorithms for doing what we need, one of the simplest is the **top-down greedy algorithm**:

1. Consider the **root** O of the tree
2. Choose the **largest instruction tile matching** O and possibly some of its siblings ($\text{munch}(O)$)
3. Let $O_1, O_2, \dots, O_k$ be the roots of subtrees which border with the covered tile
 - **Recursively call** munch on $O_1, O_2, \dots, O_k$ (in that order)
4. **Terminate** when the **entire tree has been munched**
5. **While** the algorithm **visits** the tree in depth-first order, it **appends the tiles to a list**, to produce the actual code, the list is reversed.

The algorithm is fast and operates in **linear time**, w.r.t the tree size.

We can **improve** this approach in different ways:

- Use **different cost criterion** instead of tile size
- Improve the performance by leveraging **dynamic programming**

Other code generation approaches

One alternative approach is “**peephole optimization**”. Consider some **generated suboptimal code** created using a simple approach. Using a **sliding window** on the instruction sequence, **apply pattern matching** to detect sequences of instructions that admit a replacement with a faster semantically equivalent code snippet.

To leverage **idiomatic complex instructions** (like vector instructions or packed shifts etc...), we need to have a **pre-pass and match them as early as possible**, since they would be very difficult to generate afterwards, especially with a top down algorithm.

Data-flow analysis

To perform data-flow analysis, we **start from the CFG**. We will **work at the function granularity**. Program **variables** and procedure **arguments** are **undifferentiated, and treated as temporary symbols**. The **entry point** has **no predecessor**, the **return point** has **no successor**.

An assignment to a variable is termed a definition. An occurrence of a variable in an expression is termed a use. A **node** p has **two sets of properties**: $\text{def}(p)$ and $\text{use}(p)$ **such that**:

- $p: a = a + b$ implies $\text{def}(p) = \{a\}$, $\text{use}(p) = \{a, b\}$
- $\text{read}(a)$ and $a = 7$ implies $\text{def}(p) = \{a\}$, $\text{use}(p) = \{\}$

An **instruction** may be replaced by the indication of the variables that occur in the instruction, qualified as used or defined.

We can read our CFG as a FSA where:

- Terminal **alphabet**: instruction labels $\mathcal{I}$
- The **language of strings** on $\mathcal{I}$ which label a path in the CFG from the initial state to a final state

Not all paths accepted by the CFG automaton are feasible, e.g. contradictory conditionals are ignored. Thus, **answers from static analysis are conservative approximations**.

Liveness analysis

Consider a CFG graph and an edge (p, q_1) . A variable a is **live on the edge** if it exists a path $(p, q_1, \dots, q_n)$ with $n \geq 1$ reaching q_n such that:

1. a is in $use(q_n)$
2. **None of the nodes traversed on this path defines a** (excluding p and q_n)

The node q_n may coincide with p .

If p has two successors, we say that a is **live on exit from p** if it is **live on one or the other edge**. a being live on exit from node p does **not imply that the variable is defined in p** .

a is **live on entry to node p** if it is **live on any of the edges entering the node**.

We can **formalize** the property of being **live on exit from node p** as follows:

- Consider the sets $D(a)$ and $U(a)$ respectively defining the sets defining and using a
- In the **language $L(A)$ of the CFG automaton there exists a string** (a path) of the following form:

$upvqw$

Where:

- u is a path from the initial node
- v are instructions that do not define a (may be empty)
- q is an instruction such that $a \in use(q)$
- w is a path to the final node

This forms a **regular language**.

For each node p , we can define **two equations** that:

- **Correlate the variables live on exit with that on entry** of p
- **Correlate the variables live on exit** of p to those on **entry of the immediate successor** of p .

Let $live_{in}(p)$ and $live_{out}$ be the sets of vars live on entry and exit. Let $succ(p)$, $pred(p)$ be the immediate successors and predecessors. For a final node p we have $live_{out}(p) = \emptyset$. For any other node:

$$live_{in}(p) = use(p) \cup (live_{out}(p) \setminus def(p))$$

$$live_{out}(p) = \bigcup_{\forall q \in succ(p)} live_{in}(q)$$

The **solution to the equations can be solved by iteration**, starting with the empty sets as iteration 0. We iterate substituting the unknowns until two different subsequent iterations do not differ. This is the **fixed point**, which is the **solution** to the system. To guarantee convergence, let us see if these equations respect the **convergence criterion**:

1. **Bounded cardinality**: both sets has a finite size bounded by the number of variables in the subprogram
2. **Monotonicity**: an iteration never removes a variable from the previous approximation
3. **Halt**: if an iteration lets the previous approximation unchanged, the algorithm halts

Thus the algorithm converges. The **speed of convergence is dependant on the scanning order**, the **solution** is, however, **always the same**.

Considering the following algorithm:

```

for each n
  in(n) = {}
  out(n) = {}

repeat
  for each n
    in'(n) = in(n)
    out' = out(n)
    in(n) = join(use(n), minus(out(n), def(n)))
    out(n) = for each q in succ(n) join(q)
until for each n in'(n) = in(n) and out'(n) = out(n)

```

Each join on the in/out sets takes linear time. The inner for loop computes a constant number of unions per CFG node (at most N), since there are $\mathcal{O}(N)$ nodes we have that the loop takes $\mathcal{O}(N^2)$. Each iteration must add some elements to some set, but each set has a cardinality bounded by the number of variables N . The sum of the cardinalities of all in and out sets is therefore bounded by $2N^2$. It follows that the repeat loop can iterate at most $2N^2$ times. Hence the **worst-case run time** of the algorithm is $\mathcal{O}(N^4)$.

To optimize the complexity we can choose between **two set data structures**:

- **Bit vector**: each element is a bit in an integer; union and intersection are implemented with a OR and AND respectively
 - Set operation take $N/|w|$ cycles, independently of how many values are in the set
 - Better for **dense sets**
- **Linked list**: the elements in a set are represented by a linked list. The union of two sets is realized by merging the two lists and discarding duplicated elements
 - The time to perform union is proportional to the size of the sets being united
 - Better for **sparse sets**

Generalization

The method for computing liveness can be **generalised into a general framework** for performing other kinds data-flow analysis:

- **For forward analysis**:
 - For each node in a basic block, define **two sets** $gen(p)$ and $kill(p)$
 - **Compute iteratively** the two sets:


```

out(p) = transfer(in(p), gen(p), kill(p))
in(p) = for each q immediate predecessor of p: join(q)

```

Where the $transfer()$ function is a function that combines in some way the three argument sets

- **For backward analysis**:
 - For each node in a basic block, define **two sets** $gen(p)$ and $kill(p)$
 - **Compute iteratively** the two sets:


```

in(p) = tranfer(out(p), gen(p), kill(p))
out(p) = for each q immediate successor of p: join(q)

```

E.g. liveness analysis is a backward analysis where:

- gen and $kill$ are equivalent to use and def
- $transfer(\cdot) = use(p) \cup (out(p) \setminus def(p))$

The same properties we have seen for liveness still hold.

Using this framework we can define **different kinds of analysis** such as:

- **Useless definitions:** an instruction defining a variable which is not in the live-out set is useless (dead-code), and may be deleted (provided it does not have side-effects)
- **Elimination of common subexpressions:** if an expression is computed more than once, save the result into a temporary the first time it is computed, and eliminate the subsequent computations
- **Constant folding:** if the operands of an expression are constant, do the computation at compile time, thus saving the code for computing the expressions
- **Constant propagation:** If a temporary is assigned a constant value, replace a subsequent use of the temporary with the constant

Usually analysis are done in a loop of three phases:

1. Statically analyze the program to gather information on program properties
2. Use the information to identify the instructions or instruction patterns that can be transformed
3. If a transformation improves the program, apply it
4. After the transformation, the program has changed, and static analysis may be partially invalidated, repeat the steps

Since the optimization phase may change, delete, and generate IR statements, machine-code generation phase typically comes after.

Reaching definitions

See FLC notes.

Register allocation

Access to registers much faster than access to memory, thus we want to **map the most used variables to registers**. However, the **number of live variables may exceed available registers**, meaning we **need to save variables into memory** to free the needed registers (e.g. for some instruction).

The **scopes** at which we can do register allocation is:

1. Complex expressions
2. Basic blocks
3. Procedure
4. Program

A **smaller allocation unit is easier to work with**, can be easily integrated into the instruction-selection phase and can be more suitable for dynamic compilation. **Global management of registers across procedure can improve performance, but is much more complex.** We will look at intra-procedural regalloc.

The **principles** we are working with are the following:

1. If **two variables are both live** in the **same program point**, they **cannot be stored in the same register**
2. If the **liveness intervals** of two variables are **disjoint**, the **same register can be used**

Graph coloring approach

Definition (Liveness interference relation):

A binary, symmetric relation between two variables a and b denoting the fact that the liveness intervals of a and b overlap. The interference relation can be depicted as a non-directed graph.

This relation can be **represented as a graph**, and **registers can be viewed as colors**. The assignment needs to **paint nodes of the graph with colors, so that adjacent nodes differ in color**. This is the **graph-coloring problem and is not always solvable**, we however have heuristics that are polynomial. **To solve the intractability, we spill (store) variables to memory and recompute liveness until we can color the graph.**

Algorithm outline:

- Repeat until successful
 1. **Build**: construct the Liveness Interference Graph
 - Until the graph is empty:
 1. **Simplify**: Reduce the graph removing colorable nodes
 2. **Spill**: Select from remaining nodes a candidate for spilling
 2. **Select**: Assign registers or mark nodes as spilled
 3. Start over: modify code to manage spills

Let us look more in depth at the most important procedures:

- **Simplify**:
 1. If G contains a node m with fewer than K neighbours
 2. Construct the graph $G' = G \setminus \{m\}$
 3. Push m on S
 4. Repeat until fixed point
- **Spill**:
 1. If $G = \{\}$, all remaining nodes have $\geq K$ neighbours
 2. Choose a node s and mark it as candidate for spilling
 3. Push s on S and compute $G' = G \setminus \{s\}$
 4. Retry simplify
- **Select**:
 1. For each node s in S , assign a color to it
 2. If s does not interfere with the previous popped node, reuse the same color
 3. As long as s is not marked for spilling, it is guaranteed to be colorable
 4. If s is a spilling candidate, either the node can be colored or the potential spill is marked as an actual spill

WCET is $\mathcal{O}(n^3)$, but in average is almost quadratic (most expensive operation is building the live interference graph).

Linear scan

Guarantees linear time.

```
def LinearScan:
  active = {}
  for each live interval i, in order of increasing start point:
    ExpireOldIntervals(i)
    if length(active) = R:
      SpillAtInterval(i)
    else:
      register[i] = a register removed from the pool of free registers
      active += {i} // sorted by increasing end point

def ExpireOldIntervals:
  for each interval j in active, in order of increasing end point:
    if endpoint[j] >= startpoint [i]:
      return
  else:
    active -= {j}
    add_to_free_pool(register[j])

def SpillAtInterval:
  spill = active.tail
  if endpoint[spill] > endpoint[i]:
    register[i] = register[spill]
    location[spill] = get_new_stack_location()
    active -= {spill}
    active += {i} // sorted by increasing end point
  else:
    location[i] = get_new_stack_location()
```

Let V be the number of variables and R be the number of registers. Since $|\text{active}| \leq R$, if R is assumed to be smaller than V , linear scan is $\mathcal{O}(V)$. In architecture with a lot of available registers, the complexity is more $\mathcal{O}(V \times R)$.

Register allocation with SSA

SSA form makes it easy to perform interference analysis due to the **dominance property** (definitions dominate uses). **Liveness analysis should be done before ϕ -function translation.**

For each variable v , **start from each use site and walk backwards** on the CFG, **stopping when the definition site is reached**. The algorithm processes only the blocks where v is live. Thus the **running time becomes proportional to the size of the interference graph** that it constructs.

Optimization

In this sections we will explore some optimizations assuming a procedural language compiled for a hypothetical superscalar processor called S-DLX.

Characteristics of the **source language**:

1. Similar to **Fortran 90**
2. We use **do all loops** to indicate that all iterations may be **executed concurrently**
3. **Arrays are stored contiguously in memory in column-major form**
4. **All arguments are passed by reference**

The optimization are not limited to by this assumptions, it's just that these assumptions simplify the explanation.

For a compiler to apply an optimization to a program, it must do 3 things:

1. Decide a part of the program to optimize and the transformation to apply
 - Most difficult and poorly understood part
2. Verify that the transformation is acceptable
3. Transform the program
 - This will be our focus

Correctness

Definition (Legal transformation):

A transformation is legal if the **original and the transformed programs** produce exactly the **same output for all identical executions**.

Definition (Identical executions):

Two executions are identical if they are **supplied with the same input data** and if every **corresponding pair of non-deterministic operations** in the two executions produces the **same result**.

Definition (Semicommutative and semiassociative):

We call operations that are **algebraically but not computationally commutative** (or associative) semicommutative (or semiassociative).

We can now define when a transformation is legal:

Definition (Criterion for legal transformation):

A transformation is legal if, **for all semantically correct program executions**, the **original and the transformed programs produce exactly the same output for identical executions**.

For languages that define a specific semantics for exceptions (i.e. IEEE floating-point standard), **meeting the previous definition tightly constraints the permissible transformations**.

If we **assume that exceptions are only generated when the program is semantically incorrect**, a transformed program **can produce different results when an exception occurs and still be legal**. An **alternative correctness criterion** is to

consider equivalent all permutations of semicommutative operations. In practice we check whether the numeric results differ by more than a certain tolerance, and if they do, force the compiler to employ the previous definition.

Target architecture's influence

The structure of an architecture dictates how the compiler must optimize along a number of different axes: maximize the use of computational resources, minimize the numbers of operations, minimize the use of memory bandwidth and the size of total memory required.

Definition (Stride):

The stride is the distance in memory between consecutively accessed elements of an array

Stride can have a major performance impact, e.g. a loop accessing memory with stride-1 maximizes memory locality; bigger strides may load unrelated memory into cache and filling it up faster, ruining locality.

Another key to achieving peak performance is using as much as possible compound operations in the ISA.

Optimization phases

Let us define **three different representations**:

1. **High-level** intermediate language (HIL), very close to the original source language
2. **Low level** intermediate language (LIL), an abstract machine language
3. **Object code**

The **high-level optimization phase** begins by doing all of the **large-scale restructuring** that will significantly change the organization of the program: e.g. procedure restructuring and scalarization. **Next, high-level data-flow optimizations**, partial evaluation and redundancy elimination are performed. These optimizations **simplify the program** as much as possible.

The **main focus is loop optimization**. During **loop preparation** loops are **converted in a form more amenable to optimization** (i.e. perfect loop nests). Then **loop reordering is performed**, in which iterations are reordered to maximize parallelism and locality. Finally a **loop post-processing phase** organizes the resulting loops to minimize loop overhead.

The **low-level optimization phase** begins by converting the program to LIL. **Many of the data-flow and redundancy elimination optimizations** that were applied to the HIL are **reapplied** to the LIL. Additionally **induction variable optimizations** are performed. A **final LIL optimization pass applies procedure call optimizations**.

Code generation converts LIL into assembly language and low-level optimizations are applied.

Finally **object code is generated by the assembler**. After profiling, profile-based cache optimizations can be applied to the executable program itself.

Dependence analysis

Dependence analysis identifies dependency constraints, which are then used to determine whether a particular transformation can be applied without changing the semantics of the computation.

Definition (Dependence relation):

A dependence is a **relationship between two computations** that places **constraints on their execution order**.

We denote an unspecified type of dependence between statement $S1$ and $S2$ by $S1 \implies S2$.

We have **two types** of dependencies:

- **Control**: There is a control dependence if $S1$ defines whether $S2$ will be executed
- **Data**: There is a data dependency if two statements cannot be executed in parallel because they conflict on some variable
 - 3 types:
 - * RAW or flow dependence
 - * WAR or anti-dependence
 - * WAW or output dependence
 - Storage replication can allow statements with an anti-dependence or output dependence to execute concurrently

To **capture the dependence information** for a piece of code, the compiler creates a **dependence graph**. Each node of this graph represents one statement and each arc indicates a dependence.

Because it can be cumbersome to account for both control and data dependences during analysis, **sometimes compilers convert control dependences into data dependences** using a technique called **if-conversion**.

Definition (if-conversion):

Introduces additional boolean variables that encode the conditional predicates; every statement whose execution depends on the conditional is then modified to test the boolean variable. In the transformed code, **data dependence subsumes control dependence**.

Dependencies can be **loop-carried**: dependencies that exists between iterations of a loop. To compute dependence information for loops **the compiler must analyse the subscript expressions**: it is sufficient to **determine whether any of the iterations can write a value that is read or written by any of the other iterations**. The dependence analysis algorithms may require that the loops have only unit increments. When they do not, the compiler may be able to normalize them.

Definition (Generalized perfect nest of d loops):

```
do i_1 = l_1, u_1
  do i_2 = l_2, u_2
    ...
    do i_d = l_d, u_d
      a[f_1(i_1, ..., i_d), ..., f_m(i_1, ..., i_d)] = \
        a[g_1(i_1, ..., i_d), ..., g_m(i_1, ..., i_d)]
    end do
    ...
  end do
end do
```

An **iteration can be uniquely named by a vector** of d elements $I = (i_1, \dots, i_d)$ where each index i_p stays within bounds.

Definition:

We say that iteration J was executed before I and we write $I < J$ iff $\exists p : (i_p < j_p \wedge \forall q < p : i_q = j_q)$.

A **reference in some iteration J depends on a reference in iteration I** if and only if **at least one reference is a write and $I < J \wedge \forall p : f_p(I) = g_p(J)$** . In other words, **there is a dependence when the values of the subscripts are the same in different iterations**.

When $X \Rightarrow Y$ we can **define the dependence distance** as:

$$Y - X = (y_1 - x_1, \dots, y_d - x_d)$$

When **all the dependence distances for a specific pair of references are the same**, the potentially unbounded set of dependences can be represented by the dependence distance, which in this case is **called distance vector**.

The **first non-zero element** of the distance vector **must be positive**. If it was negative, this would indicate a dependence on a future iteration, which is impossible.

In some cases it is not possible to determine the exact dependence distance at compile-time, or the dependence distance may vary between iterations; but there is enough information to partially characterize the dependence. For these cases we use the **direction vector**:

$$W = (w_1, \dots, w_d) \text{ where } w_p = \begin{cases} < & i_p < j_p \\ = & i_p = j_p \\ > & i_p > j_p \end{cases}$$

General purpose transformations

Data-flow based loop transformations

Loop-based strength reduction: Reduction in strength **replaces an expression with one that is equivalent but uses a less expensive operator**. Strength reduction can be applied in a loop: whenever the compiler finds a reducible expression, it will **initialize a temporary** with an expression, **substitute** the expression with another less expensive one and **update the temporary each at iteration**.

The most common use of strength reduction is **strength reduction of induction variable expressions**.

Loop-invariant code motion When a computation appears inside a loop, but its **result does not change between iterations**, the compiler can **move that computation outside the loop**. It can be applied **both at a high-level to expressions or at a low-level to address computations**. The precomputed value is generally assigned to a register.

Code hoisting is a general term referring to any transformation that moves a computation to an earlier point in the program. **Loop-invariant code motion is one form of hoisting**.

Loop unswitching Loop unswitching is **applied when a loop contains a conditional with a loop-invariant test condition**. The **loop is replicated inside each branch of the conditional**, saving the overhead of conditional branching inside the loop. If there is any chance that the condition evaluation will cause an exception, it must be guarded by a test that the loop will be executed.

Loop reordering

In this section we describe transformations that **change the relative order of execution of the iterations** of a loop nest or nests, in order to expose parallelism and improve memory locality.

Loop interchange **Loop interchange exchanges the position of two loop in a perfect loop nest**. It may be performed to enable/improve vectorization, improve parallel performance, reduce stride and increase the number of loop-invariant expressions in the inner loop.

Interchanging loops is legal when the altered dependences are legal and the loop bounds can be switched. If loops p and q in a perfect loop nest of d loops are interchanged, each dependence vector $V = (v_1, \dots, v_p, \dots, v_q, \dots, v_d)$ in the original nest becomes $V' = (v_1, \dots, v_q, \dots, v_p, \dots, v_d)$ in the transformed loop nest. If V' is lexicographically positive, then the dependence relationships of the original loop are satisfied. Switching the loop bounds is straightforward when the iteration space is rectangular. When it's not, computing the bounds is more complex. Further techniques are necessary to manage imperfectly nested loops.

Loop skewing Primarily useful in combination with loop interchange. It was **invented to handle wavefront computations**, so called because the updated to the array propagate like a wave across the iteration space.

Skewing is **performed by adding the outer loop index multiplied by a skew factor, f , to the bounds of the inner iteration variable, and then subtracting the same quantity from every use of the inner iteration variable inside the loop**. Because it alters the loop bounds but then alters the uses of the corresponding index variables to compensate, skewing does not change the meaning of the program and is **always legal**.

Loop reversal **Reversal changes the direction** in which the loop traverses its iteration range.

If loop p in a nest of d loops is reversed, then for each dependence vector V , the entry v_p is negated. The reversal is legal if each resulting vector V' is lexicographically positive.

Strip mining Strip mining is a method of adjusting the granularity of an operation.

It **divides the loop into n strips of k granularity**. Then we **execute the loop one each strip at a time**. Finally, we might have to do some **cleanup in case that n does not divide k** .

One of the most common uses of strip mining is to choose the number of independent computations in the innermost loop of a nest. On a vector machine, the serial loop can then converted into a series of vector operations.

Cycle shrinking Cycle shrinking is essentially a **specialization of strip mining**. When a loop has dependences that prevent it from being executed in parallel, the compiler may still be able to expose some parallelism if the dependence distance is greater than one. Cycle shrinking will convert a serial loop into an outer serial loop and an inner parallel loop.

Loop tiling Tiling is the **multidimensional generalization of strip mining**. It is primarily used to improve cache reuse.

Loop distribution Distribution breaks a single loop into many. Each of the new loops has the same iteration space as the original, but contains a subset of the statements.

It can be used to: create perfect loop nests; create subloops with fewer dependences; improve instruction cache and instruction TLB locality due to shorter loop bodies; reduce memory requirements by iterating over fewer arrays; increase register reuse by decreasing register pressure.

Loop fusion Fusion is the **inverse transformation of distribution**.

For two loops to be fused, they must have the same loop bounds (it is sometimes possible to make them identical by peeling). Two loops may be fused if there do not exist statements S_1 in the first loop and S_2 in the second such that they have a dependence $S_2 \implies S_1$ in the fused loop. The reason is that before fusing, all instances of S_1 execute before any S_2 .

Loop restructuring

This section describes loop transformations that **change the structure of the loop, but leave the computations** performed by an iteration and their relative order **unchanged**.

Loop unrolling Unrolling replicates the body of a loop a number of times u , called the unrolling factor, and iterates by step u instead of step 1.

It is a **fundamental technique for generating the long instruction sequences required by VLIW machines**. It can improve performance by reducing the loop overhead, increasing ILP and improving cache/register locality.

Unrolling **can be applied to any loop** and can be done profitably at both the high and the low levels. **Some compilers also perform loop re-rolling prior** to unrolling because programs often contain loops that were unrolled by hand.

When the it is not known at compile time whether the iteration number will be a multiple of u , a loop epilogue must be emitted. If $u > 2$ the epilogue itself is a loop.

Most compilers for high-performance machines will **unroll at least the innermost loop of a nesting**. **Outer loop unrolling is not as universal** because it yields **replicated instances of the inner loops**. To avoid the additional control overhead, the **compiler can often fuse the copies back together**. This combination of transformations is sometimes referred to as **unroll-and-jam**.

Loop quantization Loop quantization is **another approach to unrolling** that avoids replicated inner loops. Rather than creating multiple copies and then subsequently eliminating them, quantization **adds additional statements to the innermost loop directly**. The iteration ranges are changed, but the structure of the loop nest remains the same.

Loop coalescing Coalescing **combines a loop nest into a single loop, with the original indices computed from the resulting single induction variable**. It is **always legal** since it does not change the iteration order.

Loop collapsing Collapsing is a simpler, more efficient, but less general version of coalescing in which the **number of dimensions of the array is actually reduced**. Collapsing **eliminates the overhead of multiple nested loops and multidimensional array indexing**.

It is best suited to loop nests that iterate over memory with a constant stride.

Loop peeling A small number of iterations are removed from the beginning or end of the loop and executed separately. If only one iteration is peeled, the code for that iteration can be enclosed within a conditional. For a larger number of iterations, a separate loop can be introduced.

Peeling has two uses:

1. **Removing dependences** created by the first or last few loop iterations
 - Enables **parallelization**
2. **Matching the iteration control of adjacent loops**
 - Enables **fusion**

Peeling can be **applied to any loop** since it simply breaks a loop into sections without changing the iteration order.

Loop normalization Normalization **converts all loops so that the induction variable is initially 1 (or 0) and is incremented by 1 on each iteration**. It can expose opportunities for fusion and simplify inter-loop dependence analysis.

Loop spreading Spreading takes **two serial loops and moves some of the computation from the second to the first so that the bodies of both loops can be executed in parallel**.

The number of iterations by which the body of the second loop must be delayed is the maximum dependence distance between any statement in the second loop and any statement in the first loop, plus 1 (to ensure there are no dependence within an iteration).

Spreading is primarily beneficial for exposing instruction-level parallelism.

Loop replacement transformations

This section describes loop transformations that operate on whole loops and completely alter their structure.

Reduction recognition While a loop with direction vector ($<$) must normally be executed serially, **reductions can be parallelized if the operation performed is associative**. **Commutativity provides additional opportunities for reordering**. Maximum parallelism is achieved by computing the reduction with a tree: pairs of elements are summed, then pairs of these results are summed, and so on. The number of serial steps is reduced from $\mathcal{O}(n)$ to $\mathcal{O}(\log n)$.

Loop idiom recognition Parallel architectures often provide **specialized hardware that the compiler can take advantage of**. Frequently, for example, SIMD machines support **reduction directly in the processor interconnection network**. Some parallel machine include hardware also for **parallel prefix operations**.

Array statement scalarization When a **loop is expressed in array notation, the compiler can either convert it into vector operation or scalarize it into one or more serials loops**. However, the conversion is not straightforward because array notation requires that the operation be performed as if every value on the right-hand side and every sub-expression on the left-hand side were computed before any assignments are performed.

Memory access transformations

Array padding **Padding is a transformation whereby unused data locations are inserted between the columns of an array or between arrays. Padding is used to ameliorate a number of memory system conflicts**.

Cached memory systems, especially those that are set-associative, are less sensitive to low power-of-two strides. However, large power-of-two strides will cause extremely poor performance due to cache set and TLB set conflicts. Set and bank conflicts can be caused by a bad stride over a single array, or by a loop that accesses multiple arrays that all align to the same set or bank. Thus padding can be inserted between columns of an array (intra-array padding), or between arrays (inter-array padding). The disadvantages of padding are that it increases memory consumption and makes the subscript calculations for operations over the whole array more complex.

Scalar expansion Loops often contain variables that are used as temporaries within the loop body. Such variables will create an antidependence from one iteration to the next, and will have no other loop-carried dependence. **Allocating one temporary for each iteration removes the dependence and makes the loop a candidate for parallelization.**

Scalar expansion is a **fundamental technique for vectorizing compilers**. An alternative for **parallel machines** is to use **private variables**, where each processor has its own instance of the variable. **If the compiler vectorizes or parallelizes a loop, scalar expansion must be also performed for any compiler-generated temporaries in a loop.**

Array contraction If the **iteration variable** of the p-th loop in a **loop nest is being used to index the k-th dimension** of an array x, then dimension k **may be removed** from x if:

- Loop p is parallel
- All distance vectors V involving x have $v_p = 0$
- x is not used subsequently (i.e. it is dead after the loop)

Scalar replacement When a **frequently referenced array element is invariant** within the innermost loop, it can be **loaded into a scalar** (presumably a register) before the inner loop and, if it is modified, stored after the inner loop.

Code co-location Code co-location **improves memory access behaviour by placing related code in close proximity**. An estimate is made of the frequency with which each arc in the control flow graph will be traversed during program execution using either profiling information or static estimates.

Procedure inlining can also affect code locality.

Displacement minimization The target of a branch or a jump is usually specified relative to the current value of the program counter (PC). **If control is transferred to a location outside of the range of the offset, a multi-instruction sequence or long-format instruction is required to perform the jump**. Given the cost of long displacement jumps, the **code should be organized to keep related sections close together** in memory, in particular those sections executed most frequently.

Partial evaluation

Partial evaluation refers to the general technique of performing part of a computation at compile time.

Constant propagation One of the most important optimizations that a compiler can perform. It **substitutes all uses of a variable with constant value with the constant itself**.

Constant folding When an expression contains an **operation with constant values as operands**, the compiler can **replace the expression with the result**. Typically constants are propagated and folded simultaneously.

Copy propagation Optimizations may cause the same value to be copied several times. The compiler can **propagate the original name of the value and eliminate redundant copies**.

Forward propagation Forward substitution is a **generalization of copy propagation**. The **use of a variable is replaced by its defining expression**, which must be live at that point. Substitution can change the dependence relation between variables or improve the analysis of subscript expressions in loops.

Algebraic simplification The compiler can **simplify arithmetic expressions** by applying algebraic rules to them (e.g. multiply by zero or by one etc...).

Strength reduction Strength reduction **replaces an expensive operator with an equivalent less expensive operator**. Some identities for strength reduction below:

Expression	Reduced	Types
$x \cdot 2$	$x + x$	integer, real

Expression	Reduced	Types
x^2	$x \cdot x$	integer, real
$x^{c.5}$	$x^c \cdot \sqrt{x}$	real
$i \cdot 2^c$	$i \ll c$	integer
$(a, 0) + (b, 0)$	$(a + b, 0)$	complex
$\text{len}(\text{cat}(s1, s2))$	$\text{len}(s1) + \text{len}(s2)$	string

Superoptimizer A superoptimizer represents the **extreme** of optimization, **seeking to replace a sequence of instructions with the optimal alternative**. It does an exhaustive search, beginning with a single instruction. If all single instruction sequences fail, two-instruction sequences are searched, and so on.

Redundancy elimination

Redundancy-eliminating transformations **remove two kinds of computations**: those that are **unreachable** and those that are **useless**. A computation is **unreachable** if it is **never executed**; a computation is **useless** if **none** of the **outputs of the program are dependent on it**.

Both unreachable and useless code are often created by constant propagation and other optimizations. Unreachable-code elimination can in turn allow another iteration of constant propagation. When a compiler finds useless or unreachable code, it can remove it.

After a series of transformations, particularly loop optimizations, there are often variables whose value is never used. The unnecessary variables are called dead variables and can be removed.

Common sub-expression elimination In many cases, a **set of computations will contain identical sub-expressions**. The compiler can **compute the value of the sub-expression once, store it, and reuse** the stored result. However the compiler must take into account the current register pressure and the cost of recomputing.

Short circuiting Short circuiting can be performed on **boolean expressions**. The **value** of many binary boolean operations can be **determined from the value of the first operand**. If any any of the operands in the boolean expression have side-effects, short circuiting can change the result of the evaluation.

Procedure call transformation

These optimizations attempt to **reduce the overhead of procedure calls** by: removing the call; eliminating execution of the called procedure's body; eliminating some of entry/exit overhead; avoiding some steps when making a call.

Leaf procedure optimization A leaf procedure is one that does not call any other procedures.

The simplest optimization for leaf procedures is **that they do not need to save and restore the return address**. Additionally, if the procedure does **not have any local variables** allocated to memory, the compiler does **not need to create a stack frame**.

Cross-call register allocation Separate compilation reduces the amount of information available to the compiler about called procedures. However, when both callee and caller are available, the **compiler can take advantage of the register usage of the callee to optimize the call**.

If the callee does not need all the caller-save registers, the caller can leave values in the unused ones. Additionally, move instructions for parameters can be eliminated.

Parameter promotion When a parameter is passed by reference, the address calculation is done by the caller, but the load of the parameter is done by the callee. This wastes an instruction. More importantly, if the operand is already in a register in the caller, it must be spilled to memory and reloaded by the callee.

In general, **when the compiler can statically identify all the callers of a leaf procedure, it can expand their stack frames to include enough space for both procedures**. The leaf procedure simply uses the caller's stack frame without doing any new allocation of its own.

Procedure inlining Procedure inlining **replaces a procedure call with a copy of the body of the called procedure**. It can **almost always be done**, the **exception** being when the called procedure is **recursive**. Even then, **recursive functions may benefit from a finite number of inlined calls**.

When a call is inlined, all the overhead for the invocation is eliminated. Another reason for inlining is to improve compiler analysis and optimization. In many compilers, a loop containing a procedure call cannot be parallelized because its read-write behaviour is unknown. An alternative to inlining is to perform interprocedural analysis. However it can be costly and increase the complexity of the compiler.

The primary disadvantage of inlining is that it increases code size, in the worst case exponentially.

Procedure cloning Procedure cloning is a technique for improving optimization across procedure call boundaries. The **call sites of the procedure being cloned are divided into groups, and a specialized version of the procedure is created for each group**.

Loop pushing Loop pushing **moves a loop nest from the caller to a cloned version of the called procedure**. If a compiler does not perform vectorization or parallelization across procedure calls directly, pushing is a less general way of achieving a similar effect.

Pushing not only allows the parallelization of the loop, it also eliminates the overhead of all but one of the procedure calls.

Tail recursion elimination Tail recursion is a particularly common form of recursion. Its last act is to call itself and return the value of the recursive call. The recursion can be eliminated.

Recursive programs can be transformed automatically into tail-recursive version that can be executed iteratively, but this is not commonly performed by existing compilers for imperative languages.

Function memoization Memoization is an optimization that is **applied to side-effect free procedures**. In such cases it is **possible to cache the results of recent invocations**. When the procedure is called again with the same arguments, the cached result is used instead of recomputing it.

Scheduling

Instruction scheduling techniques are **required in VLIW machines**, and to a lesser extent in superscalars, **to exploit ILP and LLP (Loop Level Parallelism)**. We need a way to **map instructions over the machine's functional units**. This mapping must account for **time constraint and dependencies among the tasks**. Our goal, as usual, is to **minimize the execution time**.

The scheduling problem is **NP-hard**, so we will need to use **heuristics**.

Basic blocks

Within a basic blocks, by definition we **do not have control dependencies**. Thus we only need to **deal with data dependencies** and the machine's resources. Data dependencies are **usually modeled with a dependency graph**, while the resources are **modeled with a table or an automaton**.

Several heuristic algorithms have been proposed to solve the scheduling problem within BBs:

- **Instruction schedulers**: for each cycle, issue as many instructions as possible from a set of ready instructions
- **Operation schedulers**: for each instruction, issue it at the most favorable cycle

Both types can schedule instructions with **As-Soon-As-Possible or As-Late-As-Possible policies**.

Beyond basic blocks

Basic Blocks do not display, on average, a large amount of parallelism. A classic study (by Tjaden and Flynn) shows that the **degree of parallelism within Basic Blocks is less than 3 for most applications**. We need to find and **exploit other sources of parallelism**, such as **loops**, or to **create larger blocks to work on**.

Barriers are imposed to scheduling by the control flow:

- **Branch barrier**: an instruction **cannot be moved beyond a branch or join point**
 - Scheduling techniques: **trace scheduling, superblock scheduling, if-conversion**
- **Loop barrier**: an instruction **cannot be moved between two iteration of the same loop**
 - Scheduling techniques: **loop unrolling, modulo scheduling**

Trace scheduling

Trace scheduling focuses on **traces, loop-free sequences of basic blocks embedded in the control flow graph**. In simpler terms, it is an execution path which can be taken for some set of input; **the chances that a trace is actually executed depends on the input data**.

Trace scheduling **schedules traces just as basic blocks**, but **in decreasing order of execution probability**: most frequent traces are scheduled better.

Trace scheduling **cannot proceed beyond a loop barrier**. To work around this we **can use loop unrolling to extend the trace**. Unrolling isn't free, so we need not to abuse it.

Trace scheduling is **very intensive and requires a lot of bookkeeping**.

Trace scheduling **works best when we have few very common and isolated traces and the rest much less common**. If **two traces have almost the same frequency** or have a lot of overlap, the approach **doesn't have much advantages**.

Superblock scheduling

It is a **variant of trace scheduling**. It creates **more redundant code**, but **removes troublesome join points and has less bookkeeping** while having the same effect on primary traces.

It is a **good solution when more than one frequent trace is present**.

Modulo scheduling

It is an algorithm that allows us to **schedule instructions beyond the loop barrier**. It is based on the **software pipelining principle**:

- Issue an iteration before the previous one has finished
- A new iteration is issued every N cycles, and the code is rearranged so that instructions in the same position in cycle i and $i + N$ are compatible
- Actually, all iterations of the same $\text{mod } N$ share this property

LLVM framework

Let us look again at the compile stages:

0. Compiler driver: parses the CLI arguments and exposes an interface to users
1. Front-end: translate a source file in the intermediate representation
 - Completely language specific, with no generic special features for parsing and AST generation (slowly changing with MLIR)
2. Middle-end: analyze intermediate representation, optimize it
 - Does most of the optimizations
3. Back-end: generate target machine assembly from the intermediate representation
 - Usually uses its own machine-specific IR and does some optimizations since only here architecture specifics are known

LLVM's intermediate representation is LLVM-IR is the language modified by the middle-end understood by the LLVM backends. IR is **modified in pipeline stages called passes** (passes may have dependencies between them and the order is not fixed) **managed and scheduled by a pass manager**.

Passes are the elementary structure of work and must:

1. **Not be dependant on pipeline layout**, since it can change between runs due to different optimization levels/options

2. Apply transformations only if they do not change the semantic of the program

Corner cases are difficult to handle, thus compiler **algorithms must be proved** to preserve semantic. Having a common methodology helps with this: **algorithms are built by combining 3 kinds of passes**:

- **Analysis**: analyzes the IR and calculates quantities need for the transformation (e.g. finding candidates for hoisting etc...)
- **Optimization**: applies the transformation for the IR
- **Normalization**: transforms the different language constructs into a generic form usable by the analysis stage

The **design stages** for a compiler algorithm is usually as follows:

1. **Analyze** the problem
2. Make some examples
3. **Detect the common case**
4. Declare the **input format**
5. Declare the **analyses** you need
6. Design an **optimization** pass
7. **Prove** its correctness
8. **Improve the performance on the common case**, leave corner cases alone since it is better to not optimize than to break programs
 - **Be conservative!**
9. Improve the effectiveness of the algorithm by applying a **normalization**

Pass overview

A pass is a **subroutine that programmatically transforms a piece of code**. The code of the pass operates on the LLVM-IR using a **set of object-oriented APIs**. Passes can be run on-demand using `opt -passes='...'`. Some useful passes are:

- Printing the CFG: `opt -passes='view-cfg' input.ll`
- Printing the dominator tree: `opt -passes='view-dom' input.ll`

Example: running two passes one after the other: `opt -passes='mem2reg,view-cfg' input.ll`

There are **different kinds of passes**:

- `CallGraphSCCPass`: post-order visit of `CallGraph` SCCs
- `ModulePass`: visit the whole module
 - `ImmutablePass`: compiler configuration - never run
- `FunctionPass`: visit functions
- `LoopPass`: post-order visit of loop nests
- `BasicBlockPass`: visit basic blocks

Each pass kind visits particular elements of a module. Each specialization comes with some **restrictions**, e.g. a `FunctionPass` cannot add or delete functions

The IR

The IR comes in **3 flavours**:

1. **Assembly**: human readable format (`.ll`)
2. **Bitcode**: binary on-disk machine-oriented format (`.bc`)
3. **In-memory**: binary in-memory, used during compilation

To generate LLVM IR:

- Assembly: `clang -emit-llvm -S -o out.ll in.c`
- Bitcode: `clang -emit-llvm -o out.bc in.c`

The **IR assembly** language is **like a RISC-based assembly language**:

- **Few instructions**, all perfectly orthogonal
 - **Infinite virtual registers**

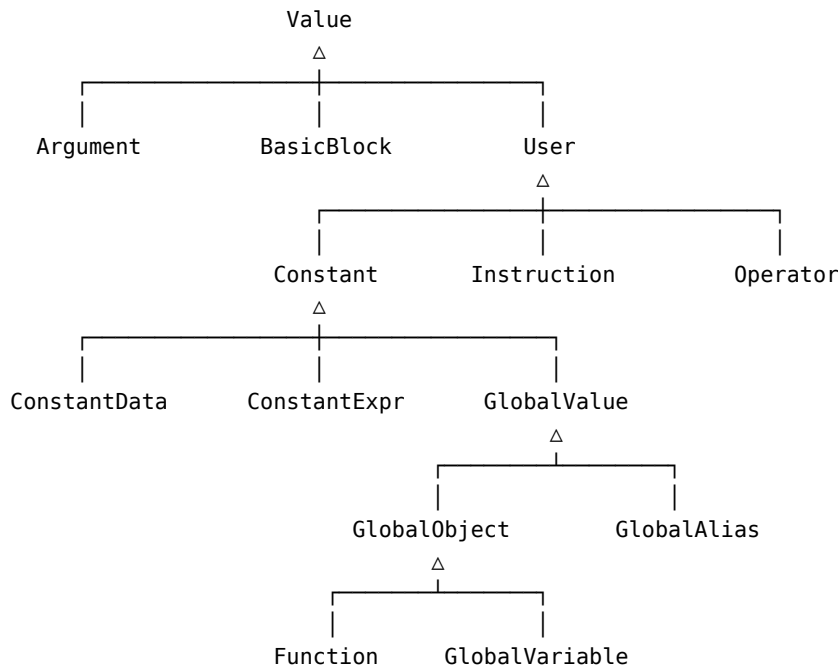
- No special-purpose registers
- No implicit flag register
- Only load and store **access memory**
- **Everything is divided in basic blocks with no implied jumps**
 - Every BB starts with a label and ends with a branch to the next BB

We have a few **high-level CISC-like instructions**, some are:

- `alloca`: used to **reserve memory** on function stacks
- `call`: **function call**
 - Calling convention is abstracted away
 - There is an implicit call stack
- `getelementptr`: pointer arithmetics

The **topmost object** of the IR is the **module**. Modules contain **globals** (global values, functions or forward declarations). **Functions** contain **basic blocks** and **arguments**. **Basic blocks** contain **instructions**. All these parts correspond directly to C++ objects.

The language is **strongly typed** with no **implicit casts**. Almost everything is typed, e.g. functions statements and registers all have types. **Objects that have a type are called LLVM values** and have `llvm::Value` as the base class.



A **variable** can be:

- **Global**: `@var = common global i32 0, align 4`
- **Function parameter**:
 - Example of function definition: `define i32 @fact(i32 %n)`
- **Local**: `%2 = load i32* %1, align 4`

The IR is **SSA-based**: every variable is statically assigned exactly once. This means that **there is no explicit register type**. The ϕ -function for handling conditional assignment is **implemented by the phi instruction**: it takes `(var_i, label_i)` pairs and it returns `var_i` if coming from `label_i`.

```

define float @max(float %a, float %b) {
    %1 = fcmp ogt float %a, %b
    br i1 %1, label %if.then, label %if.end
if.then:
    br label %if.end
if.else:
    br label %if.end
}
  
```

```

if.end:
  %2 = phi float [ %a, %if.then ], [ %b, %if.else ]
  ret float %2
}

```

Example non-trivial function:

```

define i32 @fact(i32 %n) {
entry :
  %retval = alloca i32, align 4
  %n.addr = alloca i32, align 4
  store i32 %n, i32* %n.addr, align 4
  %0 = load i32* %n.addr, align 4
  %cmp = icmp eq i32 %0, 0
  br i1 %cmp, label %if.then, label %if.end

```

```

if.then:
  store i32 1, i32* %retval
  br label %return

```

```

if.end:
  %1 = load i32* %n.addr, align 4
  %2 = load i32* %n.addr, align 4
  %sub = sub nsw i32 %2, 1
  %call = call i32 @fact (i32 %sub)
  %mul = mul nsw i32 %1, %call
  store i32 %mul, i32* %retval
  br label %return

```

```

return :
  %3 = load i32* %retval
  ret i32 %3
}

```

The CFG

Remember how we described the internal structure of an LLVM-IR module:

- `llvm::Module` is a list of `llvm::GlobalValues`
- `llvm::Function` is a kind of `llvm::GlobalValue`
- `llvm::Function` is a list of `llvm::BasicBlocks`
- `llvm::BasicBlock` is a list of `llvm::Instructions`

Functions and basic blocks act like containers with STL-like accessors and iterators. Each contained element is aware of its container and can access it using `getParent()`.

In a `llvm::BasicBlock`, the `llvm::Instructions` execute in the order specified by the list. This is not true for basic blocks. The way the basic blocks are executed is implicitly described by the branches in each block. These branches describe the Control Flow Graph of the function.

Basic blocks

LLVM has a simple API for operating on the CFG. **Every CFG has an entry basic block**, which is the first executed block and the root of the graph (`llvm::Function::getEntryBlock()`). **Starting from that**, we simply **get the terminator** (`llvm::BasicBlock::getTerminator()`) and **check the operation type** (return, branch, unreachable etc). To do so we need to use LLVM's **casting functions**:

- **Static cast** of `Y*` to `X`: `X *llvm::cast<X>(Y *)`
- **Dynamic cast** of `Y*` to `X`: `X *llvm::dyn_cast<X>(Y *)`
- Is `Y*` an **instance of** `X`? `bool llvm::isa<X>(Y*)`

Example: is the Basic block a sink?

```
lvm::isa<llvm::ReturnInst>(BB.getTerminator());
```

Every BB has **one or more predecessors** (from `pred_begin(BB)` to `pred_end(BB)`) **and successors** (from `succ_begin(BB)` to `succ_end(BB)`).

Some useful methods for BBs:

- `BasicBlock *getUniquePredecessor()`
- `moveBefore(llvm::BasicBlock *)`
- `moveAfter(llvm::BasicBlock *)`
- `splitBasicBlock(llvm::BasicBlock::iterator)`

Instructions

The `llvm::Instruction` class **defines common operations**. We can retrieve its operands with `getOperand(unsigned)`. Subclasses of `Instruction` provide specialized accessors.

New instructions are **created using the class constructor, factory methods or the `llvm::IRBuilder<>` class**. **Interface is not homogeneous!** Some instructions support all methods, others support only one.

Instructions can be **inserted automatically by `IRBuilder`** (insertion point is given at `IRBuilder` instantiation); **manually by appending to a basic block or by inserting after/before another instruction**.

Every `llvm::Value` is **typed**, retrieved by `llvm::Value::getType()`. Since every instruction is a value, all instructions are typed.

Data flow

In LLVM, the **data flow generated by the various instructions is represented by a simple hierarchy**:

- **Value**: a definition, something that can be used - `llvm::Value`
 - To visit where a definition is used: `llvm::Value::use_begin()` and `llvm::Value::use_end()`
- **User**: something that can use, that accesses a definition - `llvm::User`
 - To visit the definitions that are used: `llvm::User::op_begin()` and `llvm::User::op_end()`
- **Use**: the link between the value and the user - `llvm::Use`

Since `User` inherits from `Value` but also `Instruction` inherits from `Value`, **the value produced by the instruction is the instruction itself**.

Example:

```
%6 = load i32, i32* %1, align 4
```

The `load` is described by an instance of `llvm::Instruction`. That instance, however, also represents the `%6` variable.

Normalization passes

These passes **transform data into a canonical form**, applying some transformations in the processes.

Variable promotion (`mem2reg`)

In IR representations, some `alloca` are generated. These represent stack-allocated local variables. They are generated due to the compiler's conservative approach. This makes it difficult to perform further actions due to the `load/store`.

To limit the number of number of instructions accessing memory, we need to **eliminate load/store by promoting some variables from memory to registers**. `mem2reg` **eliminates all `alloca` calls that are used only by load/store**. It is also available as a utility: `llvm::PromoteMemToReg`. **Copy propagation is performed transparently** (by the `replaceAllUsesWith` function) by the compiler.

Loops

There are different kinds of loops, some are:

- do-while loops: condition is at the end
- while loops: condition is at the beginning
- Irreducible loops: circular paths that do not follow the standard structure of loop header+back-edge.
 - Not really possible to do unless we get really creative with goto

LLVM considers only “natural loops”. A natural loop has only **one entry node**, called header, and there is a **back-edge that enter the loop header**. Under this definition, irreducible loops are not natural loops and thus LLVM’s loop-detection ignores it.

Recap on loop terminology:

- **Back-edge**: edge entering the loop header
- **Header**: loop entry node
- **Body**: nodes that can reach the back-edge source node without passing from the back-edge target node plus the back-edge target. The
- **Exiting** nodes: nodes with a successor outside the loop
- **Exit** nodes: nodes with a predecessor inside the loop

Loop simplify (loop-simplify)

Natural loops are easy to identify but not really analysis/optimization friendly. The loop-simplify pass **normalizes natural loops by**:

1. **Ensuring the loop header has a single entry edge** (loop pre-header)
2. **Ensuring the loop has a single back-edge** (latch)
3. **Ensure exits are dominated by the loop header** (creating a exit block)

This pass actually **makes the output worse, but much more simple for the compiler to work with**. It is the best definition of a normalization pass.

Loop-closed SSA (lcssa)

Keeping **SSA form is expensive with loops: any optimization involving a SSA variable defined inside the loop and used outside the loop causes a ripple effect, causing an “accidental” quadratic complexity**.

The lcssa **inserts phi instructions at loop boundaries** for variables defined inside the loop body and used outside. This guarantees isolation between optimizations performed inside and outside the loop.

Induction variable reduction (indvars)

Some loop variables are special, like counters. A generalization is induction variables.

A variable foo is a **loop induction variables if its successive values form an arithmetic progression of the form** $foo = invariant_1 * induction + invariant_2$. foo is a **canonical induction variable** if it is **always incremented by a constant amount** ($foo = foo + invariant$).

We can see that counters are canonical induction variables since they are of the form $counter = counter + stride$.

Canonical induction variables are used to drive loop execution: given a loop, the indvars pass tries to find its canonical induction variable, normalizing it to be initialized by 0 and incremented by 1 at each iteration.

Analysis passes

Analysis allows to **derive information and properties of the input and verify those properties**. Keeping information analysis is expensive, thus algorithms updates information incrementally when transformations invalidates it.

We will see the following passes:

Pass	Name
Dominator tree	domtree
Post-dominator tree	postdomtree
Loop information	loops
Scalar evolution	scalar-evolution
Alias analysis	-
Memory SSA	memoryssa

Control flow graph

The control flow graph is implicitly maintained by LLVM, we do not run passes to build it.

Dominator tree (domtree) and post-dominator tree (postdomtree)

Dominance trees **answer to control-related queries**:

- “Is this BB **executed before** that?” - **Dominator** tree
 - Used with `llvm::DominatorTree`
- “Is this BB **executed after** that?” - **Post-dominator** tree
 - Used with `llvm::PostDominatorTree`

The interface is similar between the two:

- `bool dominates(X*, X*)`
- `bool properlyDominates(X*, X*)`

With X being either `llvm::BasicBlocks` or `llvm::Instructions`.

Loop information (loops)

Loop information is represented using two classes:

- `llvm::LoopInfo`: result of `llvm::LoopAnalysis` performed on a given function
- `llvm::Loop`: represents a single loop in a function

Using `llvm::LoopInfo` it is **possible to navigate through top-level loops** (`llvm::LoopInfo::begin()`, `llvm::LoopInfo::end()`) or get a loop for a given basic block: `llvm::LoopInfo::operator[](llvm::BasicBlock *)`

Loops are **represented in a nesting tree**. We can iterate on children loops using `llvm::Loop::begin()` and `llvm::Loop::end()`. The parent loop is retrieved by `llvm::Loop::getParentLoop()`.

We can also **query the various loop components**:

- `llvm::Loop::getLoopPreheader()`
- `llvm::Loop::getHeader()`
- `llvm::Loop::getLoopLatch()`
- `llvm::Loop::getLoopExiting()`,
- `llvm::Loop::getExitingBlocks(...)`
- `llvm::Loop::getExitBlock()`
- `llvm::Loop::getExitBlocks(...)`

Or the **basic blocks that make up the loop** with `llvm::Loop::block_begin()` and `llvm::Loop::block_end()`, or `std::vector<llvm::BasicBlock *> &llvm::Loop::getBlocks()`.

Scalar evolution (scalar-evolution)

The SCAlar EVolution pass **analyzes scalar expressions** inside loops. All expressions are **categorized and represented uniformly**, allowing it to handle general induction variables. SCEV is also useful outside of loops.

We can run SCEV with opt by passing the following flags: `-analyze -scalar-evolution`.

Printing an expression analyzed by SCEV we get the following output:

`{A,B,C}<FLAG>...<%D>`

Where:

- A is the starting value
- B is the operator
- C is the stride
- FLAG indicates some property that the framework has been able to deduce:
 - nuw: No Unsigned Wraparound
 - nsw: No Signed Wraparound
- D is the loop head BB label

SCEVs are modeled by the `llvm::SCEV` class, with a subclass for each kind of SCEV (e.g. `llvm::SCEVAddExpr`). Instantiation is disabled. A **SCEV is a tree of SCEVs**: `{(80 + %bar),+,80}` is divided into two nodes `{%1,+,80}` and `%1 = 80 + %bar`, which will form another SCEV sub-tree. The **tree leaves can be either**:

- **Constant**: `llvm::SCEVConstant`
- **Unknown** (i.e. not further splittable): `llvm::SCEVUnknown`

The `llvm::ScalarEvolutionAnalysis` pass computes all the SCEVs for a given `llvm::Function`. The pass produces an instance of the `llvm::ScalarEvolution`, which provides the following services:

- Get the SCEV representing a value: `getSCEV(llvm::Value *)`
- Get important SCEVs from other structures or SCEVs, examples:
 - `getBackedgeTakenCount(llvm::Loop *)`
 - `getPointerBase(llvm::SCEV *)`
- Create new SCEVs explicitly, examples:
 - `getConstant(llvm::ConstantInt *)`
 - `getAddExpr(llvm::SCEV *, llvm::SCEV *)`

Alias analysis

Let X be an **instruction accessing a memory location**: **is there another instruction accessing the same location?** **Alias analysis (AA)** tries to answer this question the best it can. Alias analysis is **used in a lot of memory optimizations, but it is not very accurate and often fails**.

Alias analysis is actually a **chain of multiple analyses**, executed in sequence:

- First: Basic alias analysis (`basicaa`)
- nth: Type-based alias analysis (`tbaa`)
- Last: Dummy alias analysis (`noaa`)

Every analysis in the chain fills the gap left by the previous analyses.

The main interface for AA is `llvm::AAResults`. The basic building block is `llvm::MemoryLocation`: it encapsulates a `(addr, size)` tuple and can be computed from a `llvm::Value`.

Low-level interface Given two memory locations X and Y , the analyzer **classifies them as**:

- `llvm::AliasResult::NoAlias`: X and Y are different memory locations
- `llvm::AliasResult::MustAlias`: X and Y are equal - i.e. they point to the same address
- `llvm::AliasResult::PartialAlias`: X and Y partially overlap - i.e. they point to different addresses, but the pointed memory areas partially overlap
- `llvm::AliasResult::MayAlias`: the analyzer is unable to compute aliasing information yet

Queries are performed using `llvm::AAResults::alias(X, Y)`.

A **different categorization** involves whether an **instruction I reads and/or modifies a memory location X** :

- `llvm::ModRefInfo::NoModRef`: the access neither references nor modifies the value stored in the memory location
- `llvm::ModRefInfo::Ref`: the access may reference the value stored in the memory location
- `llvm::ModRefInfo::Mod`: the access may modify the value stored in the memory location
- `llvm::ModRefInfo::ModRef`: the access may reference and may modify the value stored in the memory location

Queries are performed using `llvm::AAResults::getModRefInfo(I, X)`.

Mid-level interface To compute all aliases of a single value X LLVM provides the `llvm::AliasSet` class:

1. Instantiate a new `llvm::AliasSetTracker` starting from `llvm::AAResults`
2. It builds (one or more) `llvm::AliasSet` through a call to `llvm::AliasSetTracker::getAliasSetFor(llvm::MemoryLocation&)`

Once you have the `llvm::AliasSet` you can inspect the list of memory locations in it with the standard C++ iterator pattern.

Alias sets return memory reference and aliasing information just like the low-level interface. This information is less precise, as it is derived by conservatively aggregating more detailed data!

- `llvm::AliasSet::isRef()`: true if the memory is accessed in read-mode
 - e.g. a load is inside the set
- `llvm::AliasSet::isMod()`: true if the memory is accessed in write-mode
 - e.g. a store is inside the set
- `llvm::AliasSet::isMustAlias()`: all pointers in the set `MustAlias` with each other
- `llvm::AliasSet::isMayAlias()`: at least one pair of pointers is not a `MustAlias` pair

Memory SSA (memoryssa)

The `llvm::MemorySSAAnalysis` pass wraps alias analysis to answer queries in the following form:

Let `%foo` be an instruction accessing memory; **which preceding instructions does `%foo` depend on?**

This is done by **representing all memory accesses in a SSA-like form**:

- store-like instructions become definitions (`MemoryDef`)
- load-like instructions become uses (`MemoryUse`)
- stores to the same location in parallel CFG branches become phis (`MemoryPhi`)

MemorySSA “instructions” are owned by `llvm::MemorySSA` objects and they are overlaid on top of the normal CFG. Queries are done with:

- `AccessList *getBlockAccesses(BasicBlock *)`
- `DefsList *getBlockDefs(BasicBlock *)`

The basic interface is quite difficult to use, the basics are:

- `llvm::MemorySSAWalker` provides support for the most common queries
- `MemoryAccess *getClobberingMemoryAccess(...)` returns the nearest dominating memory access that clobbers the same memory location given

Linker

The linker is the component of the compiler toolchain that links together different object files, resolving the various data/function names, producing a single executable.

ELF crash-course

For most UNIX-like platform, object files are in the **ELF format**. It is divided into **several sections**:

- `.text`: The actual executable **code**
- `.data`: Global, **initialized writeable data**
- `.bss`: Global, **non-initialized writeable data**
 - **Not stored in the file**, since it is all zeroes
- `.rodata`: Global **read-only data**
- `.symtab`: The **symbol table**
- `.strtab`: **String table** for the **symbol table**
- `.rela.section`: The **relocation table** for section `.section`
- `.shstrtab`: **String table** for the **section names**

Symbols

A symbol is a **label for a piece of code or data**. Each symbol is **composed of**:

- `st_name`: the **name** of the symbol (stored in `.strtab`)
- `st_shndx`: index of the **containing section**
 - If `st_shndx` is 0, the symbol is not defined in the current translation units. If the linker can't find it in any TUs it will complain.
- `st_value`: the symbol's **address/offset in the section**
- `st_size`: the **size** of the represented object
- `st_info`: the symbol **type and binding**
- `st_other`: the symbol **visibility**

If `st_shndx` is `COM (0xffff2)`, it's a common symbol. **Common symbols** are used by **uninitialized global variables** and can have **multiple definitions in different translation units (TUs)**. They can also have **different sizes**, then the largest will be chosen. Common symbols have **no storage associated and usually end up in `.bss`**.

Symbols are of different **types**:

- `STT_OBJECT`: a **global variable**
- `STT_FUNC`: a **function**
- `STT_SECTION`: a **section**
- `STT_FILE`: a **TU**

The **binding** of a symbol determines **if and how it can be used by other TUs**:

- `STB_LOCAL`: **local** to the TU (static in C terms)
- `STB_GLOBAL`: **available to other TUs** (extern in C terms)
- `STB_WEAK`: like `STB_GLOBAL`, but **can be overridden**

Visibility determines whether it is **available to other modules**, where a **module is an external executable or dynamic library**. This means that visibility **determines whether a function is exported by the library**, executables usually ignore visibility. The different types of visibility are:

- `STV_DEFAULT`: **visible**, can **use an external version**
- `STV_HIDDEN`: **not visible**, always **use own version**
- `STV_PROTECTED`: **visible**, but **always use own version**

Relocations

Relocations are the most important operations done by the linker. They are **directives** where we basically **ask the linker to write the value of a certain symbol, in a certain location, in a certain way**. Relocations are **organized in relocation tables**, possibly one for section. A relocation is **composed of** the following fields:

- `r_offset`: **where** to write (as an offset in the section)
- `r_info`: **symbol identifier and relocation type**
 - **Part of this fields is to specify how to write the symbol's value**. There are several arch-specific relocation types like:
 - * `R_X86_64_64`: full symbol value (64 bit)
 - * `R_X86_64_PC32`: offset from the relocation target (32 bit)

Some linkers can also do “back-patching”, i.e. optimize possibly not efficient placeholder assembly put in by the compiler.

- `r_addend`: **value to add to the symbol's value** (optional)

Linkers more in depth

The linker is not usually invoked directly, the compiler driver does it for us. Under UNIX-like platforms, we have 4 main linkers:

- `ld.bfd`: High compatibility (even with ancient formats), the slower of the bunch

- `ld.gold`: ELF-only, optimized
- `lld`: LLVM-based and highly optimized, but with limited usage
- `gold`: the new kid in the block, built for extreme performance and parallelization

As we said before, **conceptually** the job of the linker is to **execute the following steps**:

1. Take **in several object files**
2. **Lay out the output binary**
3. **Build the final symbol table** from the inputs
4. **Apply** all the **relocations**
5. **Output the final executable/dynamic library**

To **generate the binary layout**, we **fix a starting address for each section**, **concatenate** all sections with the **same name**, all while **trying to keep similar sections close** to each other.

To **build the symbol table**, we **scan** all input tables and **merge them setting the symbol values** to their **final virtual addresses**; we **check** that all **undefined symbols have been resolved** and that **no symbol is defined twice**.

Finally, **relocations** are simply executed as specified.

Memory mapping and segments

We said the linker places similar sections together, why? Because **similar sections will be mapped together**:

- Code (e.g., `.text`) will go in a executable page
- Read-only data (e.g., `.rodata`) will go in a read-only page
- Writeable data (e.g., `.data`) will go in a writeable page

A **segment groups sections requiring similar permissions** and is **defined in the program header**. It is composed by:

- `p_offset`: offset in the file **where the segment starts**
- `p_vaddr`: **virtual address where it should be loaded**
- `p_filesz`: **size of the segment in the file**
- `p_memsz`: **size of the segment in memory**
- `p_flags`: **permission**: executable, writeable, readable

Typically a program has **two segments**: `+rx .rodata, .text, ...` and `+rw .data, .bss, ...`. The kernel reads the program headers and maps the required pages in memory with the appropriate permissions.

How come we have both `p_memsz` and `p_filesz`? Because they might differ: `.bss` is all zeros, so it's not stored in the file. The `p_memsz` portion exceeding `p_filesz` is zero-initialized.

Static linking

Static libraries `.a` are just an **archive of object files** (basically a fancy `.zip` generated by `ar`) and are **copied into the final library**. **Not all object files will be linked, only those providing undefined symbols**.

Generate an archive

```
ar rcs libmine.a example.o other.o
ranlib libmine.a
```

Link with it

```
gcc -lm -lmine main.c -o main.o # libmine must be in the library search path (e.g. /lib)
```

Dynamic linking

Dynamic linking is required to support dynamic libraries, which provide the following benefits:

- Fixing a bug in a library means multiple applications will benefit
- We need to load the library once, saving physical memory
- It does not replicate code, saving disk space

Dynamic-linking is a **lighter version of the linking process that is performed at runtime**. It loads dynamic libraries **in memory** and **provides the addresses of symbols**. It has some **differences w.r.t static linking**:

- **Used sections:**
 - `.dynsym`: **Dynamic symbol tables** (instead of `.symtab`)
 - `.dynstr`: **String table** for `.dynsym` (instead of `.strtab`)
 - `.rela.dyn`: **Dynamic relocation table** (instead of `.rela....`)
- Uses a **different set of relocation types**
- `r_offset` is **not an offset in a section, but a virtual address**

Dynamic libraries **can be anywhere in the address space**, meaning they **can't have absolute addresses at linking time**. Since we want to share read-only parts among processes, **we can't patch them at run-time!** This means that there are **no dynamic relocations** in `.text` or `.rodata`.

The solution is **compiling libraries as Position Independent Code (PIC)**. This option **forces the compiler to never use absolute addresses, but only offsets from the current PC**. In the linked binary, the **program base address is 0**. Since we always know the relative distance between symbols (even data since the distance between `.text` and `.data` is usually fixed) and the current PC, we can always calculate the addresses!

To create a dynamic library:

```
gcc -fPIC -c libone.c -o libone.o
gcc -shared -fPIC libone.o -o libone.so
```

To link against a dynamic library:

```
gcc main.c -lone -o main
```

Dynamic libraries are **not copied into the final executable**. The `.so` file must be available in predefined paths:

- `/usr/lib`, `/lib...`
- Any path in the `LD_LIBRARY_PATH` environment variable

Input object files are always linked in, their order doesn't matter. Static and dynamic libraries' order, however, is important since there may be dependencies between them. Suppose `libone.so` requires `libtwo.so`, the `-ltwo` parameter must be passed before `-lone`; or we can use fixed-point linking with `--start-group/--end-group`.

Global Offset Table and Procedure Linkage Table

What about symbols in another library, how can we access access global variables or call function in another module?

Let's first see how we can access **global variables**. For this purpose, the linker will create a **Global Offset Table (`.got`)**: It contains a **pointer-sized entry for each imported variable, holds their run-time addresses and is populated upon startup by the dynamic loader**.

For **functions**, we need another method since a program may use a lot of different function, bloating the size of the tables. Thus, **we fix the relocation only when needed (lazy loading)**. To implement it, we use **3 new sections**:

- `.plt`: **Small code stubs** to call library functions (trampoline)
- `.got.plt`: **Lazily populated GOT** for library functions addresses
- `.rela.plt`: **Relocation table** relative to `.got.plt`

For each imported function we have an entry in all of them.

At startup, `.got.plt` doesn't contain function addresses, it contains the address of the stub's instructions. This stub **invokes the dynamic loader, which will fix the relocation**. From then on, `.got.plt` will **contain the correct address**. This process is **slow** and can has some security repercussions, thus it **can be disabled by forcing the dynamic linker to resolve all function calls at load time** (by generating the object with the `-z now` option).

`.dynamic` section and stripping

The dynamic loader **ignores sections as it just knows about program headers**. In particular it **uses the `PT_DYNAMIC` header, which points to the `.dynamic` section**. `.dynamic` **contains all needed information by the loader** like needed libraries, address and size of other sections like `.dynsym`, `.dynstr`, `.got.plt`, `.rela.plt`.

This means that **the section table is optional**. The **same goes for .symtab and .strtab**. **Stripping** is the practice of **removing these sections to reduce executable footprint** (achieved with the aptly named `strip` program). Usually striped information is kept in only for debugging purposes.

Advanced linking features

1. **Relaxation**: we briefly mentioned the ability of linkers to **back-patch** the executable to correct conservative compiler decisions.
2. **Reducing size**: it might happen that unused functions make it to link time as the compiler ignores whether non-static functions are unused. The linker, however, has the information to know whether a function is effectively used or not, but it is just dumb: an **ELF section will always be linked in its entirety**. We can, however, **ask the compiler to generate a section for each function/data-object and then the linker to drop unused sections**.
3. **Link-time optimization**: basically, this means **optimizing code *after* linking instead of *before***. This enables **cross TU optimizations** due to the larger scope, **at the price of more resource usage** and a limited of optimizations possible.

With LTO, the **compiler does not emit a standard object file but a high-level internal representation** (ELF with special sections containing GIMPLE for GCC or bitcode LLVM IR for clang). The **linker needs to be able to understand these representations, merge them and optimize them**.

4. **Security**: some **useful linker options to harden** executables
 - `-z relro`: An attacker could change the `.dynamic` section for malicious purposes. With `-z relro` enabled, once `.dynamic` has been initialized, it is **marked read-only**.
 - `-z now`: An attacker could use the lazy loading system to call an arbitrary function. `-z now` **completely disables lazy loading**.
 - `-pie`: By default executable's position, unlike libraries, is not randomized. An attacker could then reuse code in the executable binary for malicious purposes. `-pie` **compiles the executable as PIC and produces a relocatable program** (similar to a shared library).

DWARF debugging standard

To provide all its debugging capabilities, a debugger should not reimplement a compiler from scratch, but it should get information from a special data structure, since reversing the compilation process is impossible. This additional data structure should be:

1. Extensible
2. Language independent
3. Compact
4. Self-contained within the executable or optionally delivered separately

This data structure, used for all UNIX platforms and even embedded ones (Windows uses its own proprietary format called CodeView), is called DWARF.

At the highest level, DWARF consists of a tree structure, with each node, called Debugging Information Entries, corresponding to an element of the program (the general idea is to mimic the compiler's AST). Every DIE has a type (tag) and a set of key-value pairs containing the debug information itself. One can examine the DIEs using the `dwarfdump` program (we will be using the LLVM version).

Describing data

Programming language symbols will not be the same as the symbols in the executable for a variety of reasons:

- Symbol names are meant for the linker, not for the debugger
- Names might have changed due to some name-mangling
- `static` globals do not have any symbol
- Stack allocated variables are not symbolicated since the linker does not care about them

Moreover, the debugger doesn't have knowledge about the format of the data types (structs, floating-points, big-endian/little-endian). Therefore DWARF needs to describe in detail all data types.

As opposed to types, variables are represented by three tags:

Apart from the obvious ones, two more tags are:

- External: encodes whether the symbol is visible from the outside
- Location: specifies where the data is located using a DWARF expression
 - There can be multiple locations since a variable can be spilled and then loaded to different registers

DWARF expressions are composed of sequences of commands in prefix notation and are interpreted by a stack-based interpreter. There are also commands for branches/function calls meaning we have a mini assembly language. This power, however, is seldom used. The most used commands are:

DWARF sections are linked together with the rest of the executable, meaning the addresses are subject to linker relocations.

All of these DIEs (along with others) support specifying where each type is declared using specific attributes:

Describing code

An obvious requirement is mapping each line of source code to assembler instructions since we want to insert breakpoints. Less obvious requirements are:

Like for data, compilation units, function calls and functions are all stored into DIEs.

Other information